

Plataforma Agenda — Contexto completo del proyecto

Preparado para Junta

30 de julio de 2026

Contents

Plataforma Agenda — Documento de contexto	1
1. Qué estamos construyendo	1
2. Stack técnico	2
3. Convenciones de arquitectura	2
4. Modelo de negocio y precios (decisiones ya cerradas)	3
5. Estado del proyecto: fases completadas	4
6. Fase en curso ahora mismo: Cobro automático	6
7. Qué queda pendiente después de esto (priorizado)	7
8. Cómo trabajamos (flujo de colaboración)	8
9. Notas operativas	8
10. Comandos útiles	8

Plataforma Agenda — Documento de contexto

Este documento resume todo lo necesario para retomar el proyecto en una conversación nueva con Claude: qué es, cómo está construido, qué se hizo, qué falta, y cómo trabajamos juntos. Está pensado para pegarse o adjuntarse al inicio de una conversación nueva.

1. Qué estamos construyendo

Plataforma Agenda es un SaaS de agendamiento, CRM y pagos para negocios de salud y bienestar en Latinoamérica — pensado como una alternativa mejor a **AgendaPro**. El nicho inicial de go-to-market es **salud especializada** (psicólogos, nutricionistas, fisioterapeutas, clínicas pequeñas/medianas), que es el hueco que AgendaPro deja mal servido (fichas clínicas genéricas, sin CIE-10, sin plantillas por especialidad).

El proyecto vive en `C:\Users\USER\Desktop\plataforma-agenda` y se desarrolla con **Claude Code** corriendo localmente en VS Code.

Diferenciadores clave frente a AgendaPro (no romper al implementar features)

1. **Precio transparente:** sin add-ons obligatorios para funciones básicas de WhatsApp/recordatorios. Si se agrega un límite de uso, debe ser generoso y visible.
2. **Permisos granulares por sede desde el día 1** (`StaffLocationRole`), no un parche posterior.
3. **Ficha de cliente configurable por vertical** vía `Client.customFields` (JSONB), no un esquema rígido.
4. **Pagos nativos con confirmación en tiempo real** (webhook), nunca con demora de días como reportan usuarios de AgendaPro.
5. **CRM predictivo**, no solo manual: la lógica de recompra/seguimiento dispara recordatorios automáticos por inactividad, no solo por cumpleaños.

2. Stack técnico

- **Next.js (App Router) + TypeScript** — un solo repo para frontend y API.
- **PostgreSQL + Prisma** como ORM (schema-first). Base de datos en Neon.
- **Tailwind CSS** para estilos.
- **Auth:** `Auth.js v5` (`next-auth`) con `@auth/prisma-adapter` y provider de `Credentials` (email + contraseña contra `User.passwordHash` con `bcryptjs`). Sesión JWT con `tenantId` y `StaffLocationRole` embebidos (solo para UI — las mutaciones re-verifican permisos contra la base de datos). **Importante:** `locationRoles` se cargan al JWT solo en el login (`authorize` callback) — nunca se refrescan en caliente. Si a un usuario le cambian el rol mientras tiene sesión abierta, necesita volver a iniciar sesión para verlo reflejado en la UI.
- **Pagos:** Stripe (internacional) + Wompi (Colombia, Bancolombia) — el modelo `Payment` contempla ambos proveedores vía el enum `PaymentProvider`. Se eligió Wompi en vez de Mercado Pago por menor fricción de cuenta/integración en el mercado colombiano inicial. Esto es para **pagos de citas de clientes finales** — separado del cobro de la propia suscripción del SaaS (ver sección 6).
- **WhatsApp:** WhatsApp Business Cloud API (Meta oficial, directa — sin Twilio ni intermediarios). El modelo `NotificationQueue` contempla el canal.

3. Convenciones de arquitectura

- Todo modelo de negocio cuelga de `tenantId` (multi-tenant). Nunca una query debe cruzar tenants sin querer — siempre filtrar por `tenantId`.
- Los permisos se validan a nivel de `locationId`, no solo `tenantId`. Un STAFF o PROFESSIONAL no debe ver datos de otra sede del mismo tenant.
- Los campos “flexibles” (ficha clínica, ficha estética, etc.) van en `Json` (`customFields`), no como columnas nuevas — evita migraciones por cada vertical.
- Nombres de archivos y componentes en inglés, textos de UI en español (mercado LatAm).
- Cada módulo nuevo lleva tests básicos cuando aplica (Vitest para lógica, Playwright para flujos críticos).

- Patrón “set-replace transaccional” para relaciones muchos-a-muchos (ProfessionalLocation, ProfessionalService, StaffLocationRole): deleteMany de lo que se quitó + upsert de lo que se mantiene/agrega, siempre validando que los ids enviados pertenezcan al tenant antes de usarlos.
- Patrón “404 en vez de 403”: cuando un usuario no tiene acceso a una sección (por rol o por plan), se le muestra 404 (notFound()), no un 403 explícito — evita filtrar información sobre qué existe.
- Las Server Actions siempre re-verifican permisos por su cuenta (no confían en que la página que las invoca ya validó todo), porque se pueden invocar directamente sin pasar por la UI.

4. Modelo de negocio y precios (decisiones ya cerradas)

- **Modelo de precios:** suscripción pura por sede/usuarios (planes fijos mensuales), **sin comisión sobre citas ni pagos procesados** — igual que AgendaPro. Se descartó un híbrido tipo Fresha (comisión sobre clientes nuevos vía marketplace) porque Plataforma Agenda no tiene ni planea un marketplace que le mande clientes nuevos a los negocios, y un costo variable sobre ingresos contradice el diferenciador #1 (precio transparente).
- **Tiers concretos** (mapeados al enum Plan ya existente en el schema):

Plan	Precio	Profesionales activos	Sedes	Incluye
INDIVIDUAL	19 USD/mes	1	1	Agenda pública+interna, recordatorios WhatsApp, pagos Stripe/Wompi, ficha por vertical
BASICO	35 USD/mes	hasta 3	1	Todo INDIVIDUAL + Reportes y comisiones
PREMIUM	59 USD/mes	hasta 8	hasta 3	Todo BASICO + CRM predictivo de recompra + Inventario

Plan	Precio	Profesionales activos	Sedes	Incluye
PRO	99 USD/mes	ilimitado	ilimitado	Todo + soporte prioritario (no forzado por código, es solo comercial)

Los límites son por **profesionales activos** (`Professional.active`), no por usuarios/logins (`StaffLocationRole`) — un STAFF de recepción con acceso no cuenta contra el tope, porque la métrica de capacidad del negocio son los profesionales que atienden, no las cuentas con login.

Nota: estos montos y topes son una propuesta razonada, ancla en precios de AgendaPro (19/29/59 USD/mes), no están validados contra costos reales (WhatsApp Cloud API, Stripe/Wompi, infraestructura) ni contra investigación de disposición a pagar del nicho — ajustar si la data real lo pide.

5. Estado del proyecto: fases completadas

Todo lo listado abajo está **hecho y verificado en vivo** (código revisado + probado en el navegador vía túnel de ngrok), salvo que se indique lo contrario.

Fase 0 — Cimientos

Schema multi-tenant + RBAC por sede. [hecho]

Fase 1 — Agenda pública 24/7 + agenda interna

Reservas públicas, agenda interna con calendario semanal, login y permisos por sede (`requireDashboardAccess`). Recordatorios automáticos por WhatsApp (Meta Cloud API directa): al reservar se encola un `NotificationQueue` (canal WHATSAPP) 24h antes de la cita; `/api/cron/send-reminders` (protegido por `CRON_SECRET`) procesa los pendientes y envía la plantilla. [hecho] hecho y verificado el encolado/manejo de errores. **Pendiente real:** el envío real de un WhatsApp todavía no se probó porque falta que Meta apruebe la cuenta de WhatsApp Business y la plantilla (“recordatorio_cita”, 3 variables) — es un trámite manual, no de código.

Fase 2 — CRM con ficha configurable por vertical

`Tenant.vertical` (GENERAL/PSICOLOGIA/NUTRICION/FISIOTERAPIA/ESTETICA) + plantillas fijas de campos por vertical, guardadas en `Client.customFields` con whitelist de keys. CRUD de clientes con historial de citas. [hecho] El constructor de campos personalizado por tenant (más allá de las 5 plantillas fijas) queda fuera de esta fase.

Fase 3 — Pagos nativos (de clientes finales, no del SaaS)

Stripe (Checkout hosted + webhook con confirmación en tiempo real) [hecho]. Wompi Colombia (Web Checkout + confirmación por API) [hecho], verificado de punta a punta con pago real en sandbox. Nota de desarrollo local: Wompi bloquea `redirect-url` apuntando a `localhost`, así que hace falta un túnel público (ngrok) para probar el checkout completo.

Fase 4 — Reportes y comisiones

Acceso restringido a OWNER/ADMIN. Filtro por rango de fechas anclado a `Appointment.startsAt`. Tres secciones: conteo de citas por estado, ingresos cobrados por proveedor+moneda (nunca sumando monedas distintas), comisión por profesional calculada sobre `Service.price` de citas COMPLETED (no sobre `Payment.amount`, para que funcione igual con pagos en efectivo). [hecho] Fuera de esta fase: exportar CSV/PDF, gráficos, comisión por servicio individual.

Fase 5 — Multi-sede avanzado, CRM predictivo, Inventario

- **Gestión de sedes:** modelo `ProfessionalLocation`, CRUD de sedes restringido a OWNER, selector de sede real en agenda interna y booking público (filtra por servicio Y sede a la vez). [hecho]
- **CRM predictivo de recompra:** regla de negocio (`isClientInactive`) — candidato si tuvo al menos una cita COMPLETED, no tiene ninguna futura, y pasaron 60+ días desde la última completada. Dos crons nuevos: `detect-inactive-clients` y `send-followup-reminders`. [hecho] verificado el encolado; el envío real de WhatsApp sigue pendiente por lo mismo que en Fase 1 (cuenta de Meta Business).
- **Inventario:** `InventoryItem/InventoryStock/InventoryMovement`. Descuento manual de stock (sin vínculo automático con citas — decisión explícita), alertas de stock bajo solo visuales. Dos niveles de permiso: cualquiera con acceso a la sede puede registrar movimientos, solo OWNER/ADMIN puede crear/editar el catálogo. [hecho]

Fase 6 (parte 1/N) — Feature-gating por plan

`src/lib/planLimits.ts` centraliza los límites (`PLAN_LIMITS`, `getPlanLimits`, `planIncludesModule`, `hasReachedLocationLimit`, `hasReachedProfessionalLimit`). Tope de sedes aplicado en `createLocationAction`. Gating de módulos (`inventory`, `reengagement`, `reports`) vía guards en `src/lib/auth-guards.ts`. Página `/plan-required` para explicar por qué algo está bloqueado. **Bloqueo duro** por `PAST_DUE/CANCELLED`: `requireDashboardAccess` redirige a `/account-locked` para TODO el dashboard interno (agenda, clientes, reportes, sedes, inventario) — la agenda pública de reservas sigue funcionando igual. [hecho] El plan y status del tenant se siguen asignando a mano en Prisma Studio en esta fase — no hay cobro automático todavía (eso es la fase en curso, ver sección 6).

Gestión de profesionales

Páginas para crear/editar profesionales (checklists de Servicios y Sedes con set-replace transaccional). Tope de profesionales activos por plan integrado aquí (`maxProfessionals`): nunca bloquea dejar/crear un profesional **inactivo**; solo bloquea si el guardado dejaría al profesional activo y eso superara el límite del plan (al editar, el conteo excluye al propio profesional editado). [hecho] No hay borrado de profesionales (solo desactivar), ni vínculo `Professional↔User` todavía.

Equipo (invitar usuarios a una sede)

Sin infraestructura de email en el proyecto, así que no es invitación por link mágico: el OWNER/ADMIN da de alta al usuario directo, con una contraseña temporal generada por el sistema, mostrada **una sola vez** vía cookie `httpOnly` de 60s (nunca en la URL). Reglas de seguridad agregadas proactivamente: un ADMIN nunca puede asignar rol OWNER ni editar a alguien que ya es OWNER en el tenant; un usuario nunca puede dejarse a sí mismo sin ningún acceso (auto-bloqueo). “Sin acceso” en todas las sedes ya cumple la función de revocar acceso — no hace falta borrar el `User`. [hecho]

6. Fase en curso ahora mismo: Cobro automático

Objetivo: automatizar el cobro mensual recurrente de la propia suscripción de un tenant a Plataforma Agenda (distinto de los pagos que sus clientes finales hacen por una cita, que ya existen). Hoy `Tenant.plan` y `Tenant.status` se asignan 100% a mano en Prisma Studio.

Decisiones de scope ya tomadas: - Solo **Stripe Subscriptions** en esta fase (Wompi no tiene un producto de suscripciones nativo — requeriría tokenización de tarjeta + un cron propio de cobro/reintentos, mucho más trabajo/riesgo). Wompi recurrente queda para una fase futura aparte. - **Sí incluir el Billing Portal de Stripe** para que un OWNER en `PAST_DUE` pueda actualizar su tarjeta y reintentar el cobro él mismo desde `/account-locked`, en vez de depender de que alguien lo arregle a mano — cierra el círculo del hard-lock que ya existe desde la Fase 6. - No hay ni habrá en esta fase self-service signup de tenants nuevos (eso sigue siendo alta manual). - No hay selector de plan en el checkout — siempre usa el Plan actual del tenant tal cual está en la base; cambiar de plan sigue siendo manual.

Ya se escribió el prompt para Claude Code: `prompt-cobro-automatico.md` (en la raíz del proyecto), con esta especificación completa:

1. Agregar `stripeCustomerId String? @unique` y `stripeSubscriptionId String? @unique` a `model Tenant` (migración aditiva).
2. Env vars nuevas: `STRIPE_PRICE_INDIVIDUAL`, `STRIPE_PRICE_BASICO`, `STRIPE_PRICE_PREMIUM`, `STRIPE_PRICE_PRO` (IDs de Price de Stripe, uno por plan — se crean a mano en el Dashboard de Stripe).
3. `src/lib/subscriptionPlans.ts` nuevo — mapea Plan a su Price ID.
4. `requireBillingAccess` nuevo en `auth-guards.ts` — a diferencia de todos los guards existentes, **no** pasa por el hard-lock de `PAST_DUE/CANCELLED` (si lo hiciera, un tenant bloqueado nunca podría llegar a la página que le permite arreglar su propio pago).

5. `src/app/dashboard/[tenantSlug]/billing/actions.ts` — dos Server Actions: `createSubscriptionCheckoutAction` (crea/reusa Customer de Stripe, crea Checkout Session en modo subscription, usa `client_reference_id: tenant.id` para poder correlacionar en el webhook) y `createBillingPortalSessionAction` (abre el Billing Portal, reusable tanto desde `/billing` como desde `/account-locked`).
6. Página nueva `/dashboard/[tenantSlug]/billing` (OWNER-only) con el estado actual y los botones de arriba.
7. Extender `/account-locked` con el botón “Actualizar método de pago”.
8. Extender el webhook existente `src/app/api/webhooks/stripe/route.ts` (mismo endpoint, mismo secreto — no crear uno nuevo) con ramas nuevas para `checkout.session.completed` en modo suscripción (guarda `customerId/subscriptionId`, pone status ACTIVE), `invoice.paid` (recupera a ACTIVE), `invoice.payment_failed` (pone PAST_DUE), y `customer.subscription.deleted` (pone CANCELLED) — sin tocar la rama existente que maneja pagos de citas de clientes finales.

Qué falta para cerrar esta fase: 1. Correr el prompt en Claude Code. 2. Yo (Claude) verificar el código y probarlo en vivo con Stripe en modo test (checkout con tarjeta 4242..., simular `invoice.payment_failed` con stripe trigger, confirmar el hard-lock, probar el Billing Portal, confirmar que los pagos de citas de clientes finales siguen intactos). 3. Documentar el resultado en `CLAUDE.md` como se hizo con cada fase anterior. 4. Vos vas a necesitar crear a mano, en el Dashboard de Stripe (modo test primero, luego modo real), 4 Products con un Price recurrente mensual cada uno, y cargar esos IDs en las env vars.

7. Qué queda pendiente después de esto (priorizado)

Siguiendo la instrucción permanente que me diste: **siempre presentar prioridades agrupadas por categoría, de la más importante a la menos importante**, empezando por la más importante.

Categoría 1 — Huecos funcionales core (ya resuelta en su mayoría esta sesión: Gestión de profesionales [hecho], Equipo [hecho]). Sin pendientes obvios de esta categoría en este momento salvo lo que surja al usar el producto.

Categoría 2 — Monetización real del SaaS - Cobro automático (Stripe) — **en curso ahora mismo**, ver sección 6. - Wompi recurrente para Colombia — deferido, fase futura. - Self-service upgrade/downgrade de plan — deferido, sigue siendo manual.

Categoría 3 — Cierre de trámites externos (no son código) - Aprobación de la cuenta de WhatsApp Business en Meta for Developers + aprobación de las plantillas “recordatorio_cita” y “seguimiento_recompra” — bloquea probar el envío real de WhatsApp en Fases 1 y 5.

Categoría 4 — Mejoras de producto no urgentes - Constructor de campos personalizado por tenant en el CRM (más allá de las 5 plantillas fijas por vertical). - Exportar reportes a CSV/PDF, gráficos, comisión por servicio individual. - Vínculo servicio↔insumo con descuento automático de inventario por cita. - Flujo de “cambiar/recuperar contraseña” (hoy no existe ninguno). - Self-service signup de tenants nuevos.

8. Cómo trabajamos (flujo de colaboración)

Este es el patrón que seguimos en cada fase, para que una conversación nueva lo replique:

1. Yo investigo el código existente (leo archivos relevantes, a veces busco información externa si hace falta) y propongo el alcance de la siguiente fase.
2. Cuando hay decisiones de producto/alcance ambiguas, te las presento como preguntas de opción múltiple **antes** de escribir código — nunca asumo.
3. Escribo un archivo `prompt-<nombre-de-la-fase>.md` en la raíz del proyecto, con esta estructura fija: Contexto, “Qué hacer” (pasos numerados con specs precisas a nivel de código), “Qué NO hacer” (límites explícitos de alcance), y “Verificación” (pasos manuales de prueba).
4. Vos le pasás ese archivo a Claude Code (corriendo local en VS Code), que implementa.
5. Me pegás el resumen que te da Claude Code.
6. Yo **nunca confío solo en ese resumen** — reviso el código directamente (leyendo los archivos) y, cuando aplica, prueba la app en vivo navegando por un túnel de ngrok con las herramientas de Chrome, porque no tengo acceso directo a la base de datos Postgres (Neon) del proyecto.
7. Solo después de verificar de forma independiente, actualizo `CLAUDE.md` con el resultado exacto — a Claude Code siempre se le indica explícitamente que no toque `CLAUDE.md`.

Restricciones permanentes que sigo: - Nunca manejo ni ingreso credenciales de pago, contraseñas ni tokens/API keys en ningún campo de la UI. - Todo prompt a Claude Code debe declarar explícitamente los límites de alcance (“Qué NO hacer”). - Todo módulo nuevo debe filtrar por `tenantId`. - Las prioridades siempre se presentan agrupadas por categoría, de más a menos importante, empezando por la más importante.

9. Notas operativas

- **Túnel de desarrollo:** `https://rival-catnip-scribe.ngrok-free.dev -> localhost:3000` (puede cambiar la URL si se reinicia ngrok). Si al navegar aparece `ERR_NGROK_8012`, significa que ngrok está arriba pero `npm run dev` no está corriendo — hay que confirmarlo y reiniciar.
- No tengo acceso directo a la base de datos Postgres (Neon) del proyecto — toda verificación de datos se hace o bien leyendo el código o bien navegando la app en vivo.

10. Comandos útiles

<code>npm install</code>	
<code>cp .env.example .env</code>	# y completar DATABASE_URL, etc.
<code>docker compose up -d</code>	# levanta Postgres local
<code>npm run db:migrate</code>	# crea las tablas a partir de prisma/schema.prisma
<code>npm run db:seed</code>	# carga datos de ejemplo (tenant, sede, servicio, cliente demo)
<code>npm run dev</code>	# <code>http://localhost:3000</code>

Este documento se generó a partir de CLAUDE.md (memoria persistente del proyecto, mantenida y actualizada por Claude tras verificación independiente de cada fase) y del historial de la conversación en la que se decidieron y construyeron las fases descritas arriba.